



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

BACHELOR OF COMPUTER SCIENCE (DATA ENGINEERING) WITH HONS.

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING (WBL)

TUTORIAL IV: GAI

GROUP: 8

PREPARED BY :

NAME	MATRIC NO
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201
SYARIFAH DANIA BINTI SYED ABU BAKAR	A23CS0183
NURUL ADRIANA BINTI KAMAL JEFRI	A23CS0258

PREPARED FOR:

DR. ARYATI BINTI BAKRI

TABLE OF CONTENT

1.0 OBJECTIVES	3
2.0 AI-ASSISTED IN DATA ENGINEERING	4
3.0 AI AGENT USED	4
4.0 USE CASE DIAGRAM	5
5.0 STEPS PROMPTED / CODE GENERATED	6
5.1 Data pipeline prompt	6
5.2 Setting up credentials	7
5.3 Transformation prompt	8
6.0 CONCLUSION RESULT	10
7.0 REFERENCES	11
Andreas Kretz. (2025, November 5). I built this complete ETL data pipeline with an AI Agent! [Video]. YouTube. https://www.youtube.com/watch?v=qGu7hDaJG1A	11
Mattia, D. P., Zhong, M., Sripada, Y., & Jaspars, M. (2025). FlowETL: an autonomous Example-Driven pipeline for data engineering. ArXiv.org. https://doi.org/10.48550/arxiv.2507.23118	11
8.0 REFLECTION	12

1.0 OBJECTIVES

This tutorial aims to explore the capabilities of AI-assisted data engineering by building a fully functional ETL (Extract, Transform, Load) data pipeline using an AI agent platform, specifically Express AI powered by Nexla. The objectives are as follows:

- To build a complete end-to-end ETL data pipeline using an AI agent (Express AI) without writing low-level code manually.
- To explore the use of AI-assisted tools in automating data engineering tasks such as API credential configuration, schema mapping, data transformation, and scheduling.
- To explore the use of AI-assisted tools in automating data engineering tasks such as API credential configuration, schema mapping, data transformation, and scheduling.
- To integrate a live weather data source (OpenWeatherMap API) with a cloud data warehouse (Snowflake) using natural language prompting.
- To evaluate the practical benefits and limitations of AI agents in a real-world data pipeline scenario, including prompt iteration and debugging.

2.0 AI-ASSISTED IN DATA ENGINEERING

Historically, constructing pipelines (also known as Data Engineering – Data Ingestion, Transformation, and Loading – ETL/ELT) manually, with a lot of boilerplate code, and manually configuring infrastructure connections is costly and takes a lot of time (Di Profio et al., 2025). Engineers have to manually navigate through graphical user interfaces (GUIs) and write custom Python, SQL or Scala scripts to bind to data sources and data warehouses.

With AI-assisted data engineering, intelligent layers are added to this lifecycle, like AI agents. Data engineers work with an AI agent, giving him natural language prompts rather than writing low level implementation code or configuring connections, field-by-field. The AI agent acts as a co-pilot that translates high-level functional requirements into operational pipelines (Steiner & Bizer, 2026) . It handles technical hurdles like inferring the schema, mapping the credentials, configuring the database connector and making simple algorithmic changes (such as converting units). This brings a transition of the data engineer's position from just tactical and syntax generation to strategic pipeline design, optimization and architecture (Gorelik, 2019).

3.0 AI AGENT USED

The AI agent employed is Express AI, a specialized natural language orchestration tool offered on express.dev, which is powered by Nexla's data engineering platform. Unlike general-purpose conversational models, Express AI works on a specific domain; it's a dedicated ETL co-pilot. It is mainly used in data engineering for automating mechanical data pipeline construction. The agent can be prompted directly via conversation, like asking it to pull data from WeatherAPI and put it inside Snowflake. On the automation front, Express AI automatically sets up API credentials, determines the source data specs, and creates secure database links without the need to navigate through tedious graphical user interfaces.

It creates intermediate "Nexsets" (virtual data products) to control the data stream, and automatically derives the underlying Python code for mid-pipeline processing, including data conversions, such as temperature from Celsius to Fahrenheit. With independent control over the scheduling of cron jobs and a high-level visualization of how data flows from the source to the

sink, Express AI frees up the data engineer from the drudgery of boilerplate coding and enables a more abstract approach to architecture validation.

4.0 USE CASE DIAGRAM

The use case diagram below illustrates the functional structure of an automated data pipeline that is deployed into the Express AI Platform (nexla.dev), and shows how a human user interacts with the various external systems. Data Engineer is the main actor who performs an action as the entire workflow depends on him/her, and his/her actions involve four use cases: Starting data pipeline setup, configuring API credentials, connecting to database, and scheduling data ingestion.

When the system is in the phase of credential configuration, it performs the dependency of <<include>> to perform the automatic inference of Schema Mapping from the stream that is received. To finish the pipeline, the engineer sets up a secure database connection to the secondary actor, Snowflake (destination), and then schedules the data ingestion in a way that actively coordinates with the WeatherAPI (source) to retrieve data on its own and with Snowflake to load the data to the target.

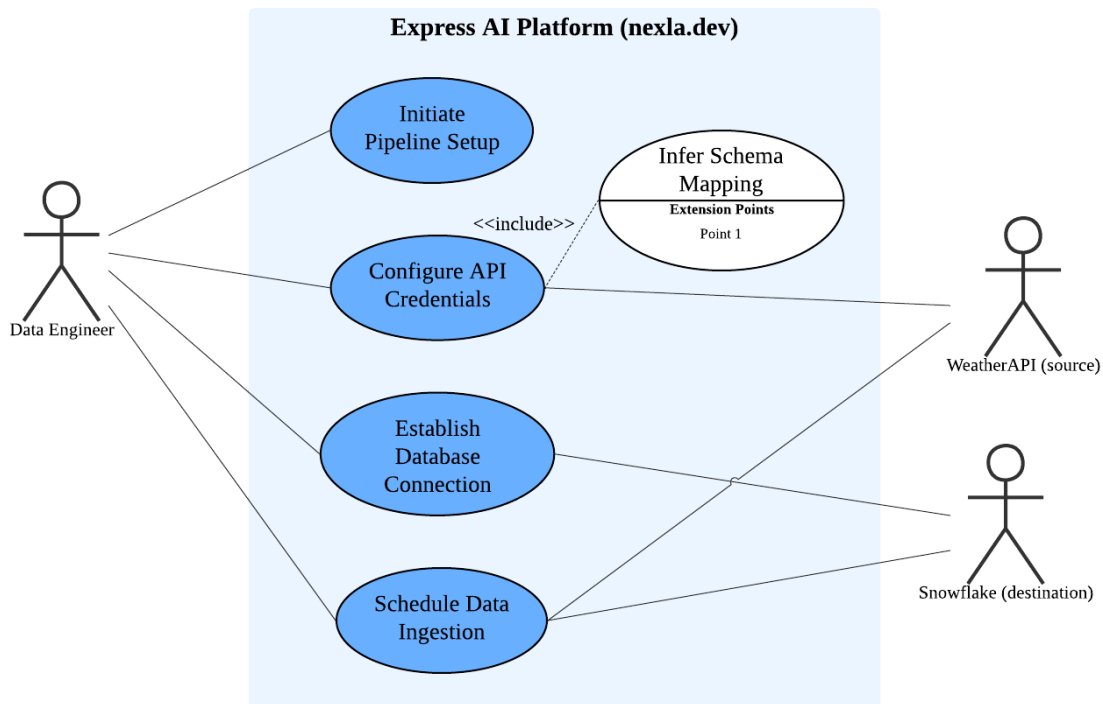


Figure 4.0 Express AI Use Case Diagram

5.0 STEPS PROMPTED / CODE GENERATED

5.1 Data pipeline prompt

The first step of this tutorial are promoting Express to create a pipeline from the weather API to Snowflake, and the pipeline should run for every 2 minutes as shown in Figure 5.1.

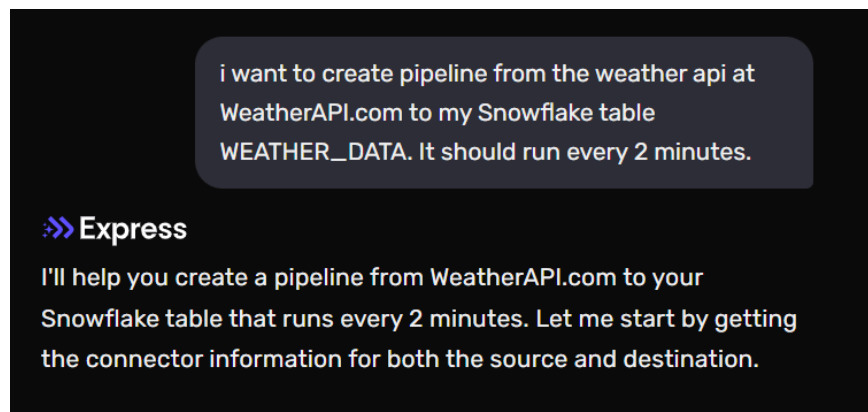


Figure 5.1 Data pipeline prompt

5.2 Credential prompt

The express asked to fill in the connection details for 2 credentials, which are OpenWeather API and Snowflake API. As shown in Figure 5.2.1 and 5.2.2 is the credentials set up prompted by the Express.

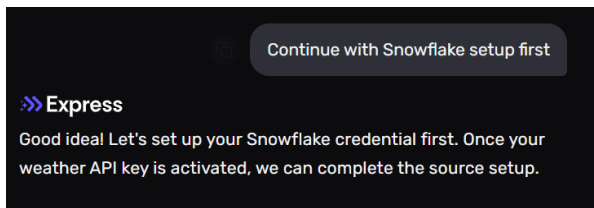


Figure 5.2.1 Prompt message Snowflake

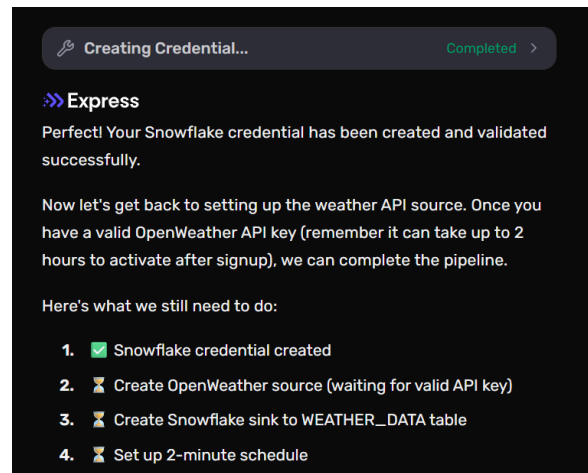


Figure 5.2.2 Express message

Then, setting up the OpenWeather API credentials is made in the Express.

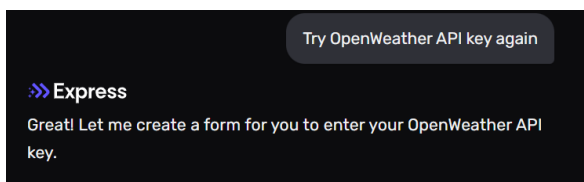


Figure 5.2.3 Prompt message OpenWeather

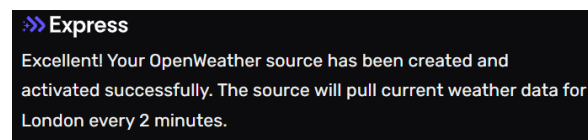
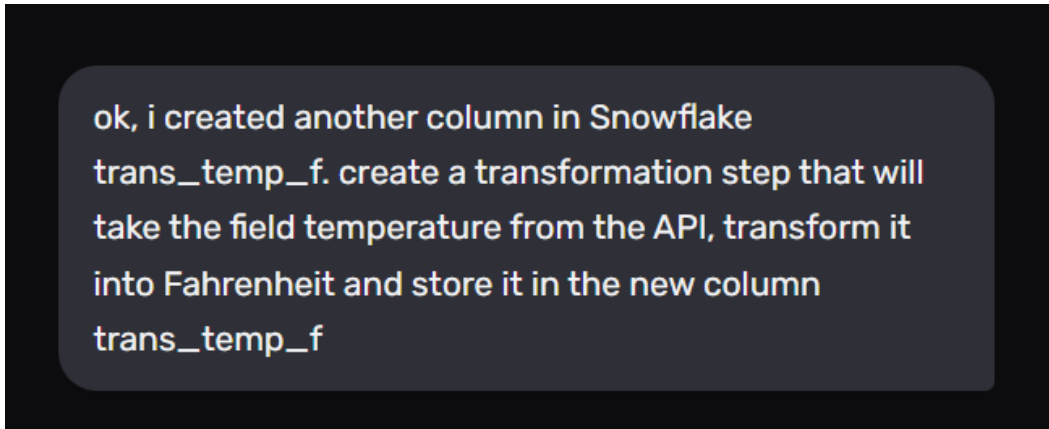


Figure 5.2.4 Express message

5.3 Transformation prompt/code

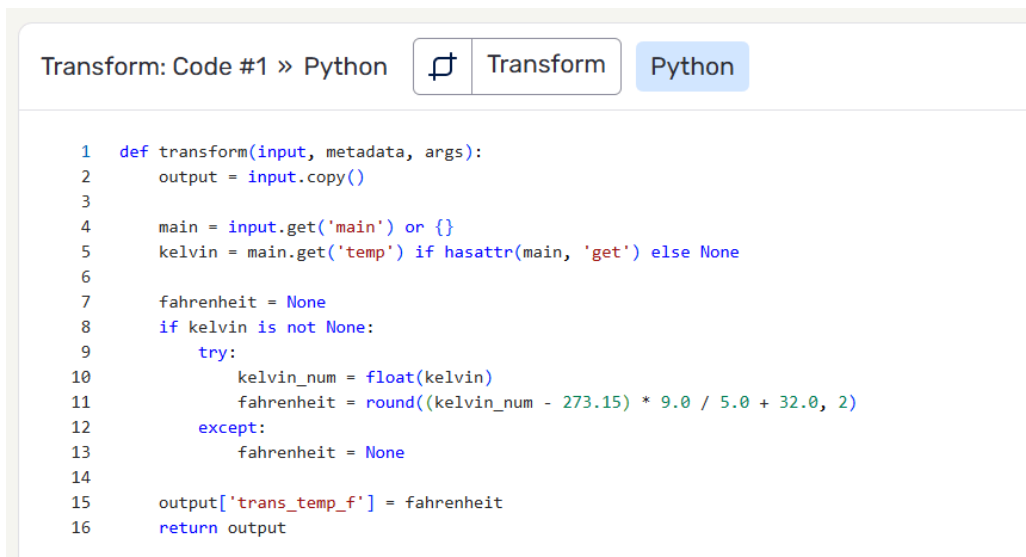
After setting up the credentials from OpenWeather and configuring the destination in Snowflake by using the API keys, the transformation phase is conducted. The example transformation prompt in Express AI is as shown in Figure 5.3.1.



```
ok, i created another column in Snowflake
trans_temp_f. create a transformation step that will
take the field temperature from the API, transform it
into Fahrenheit and store it in the new column
trans_temp_f
```

Figure 5.3.1 Transformation prompt

Then, Express AI generates a Python code transform to convert temperature from Kelvin to Fahrenheit. The code operates by accessing the nested field main.temp (temperature in Kelvin). Then, apply the conversion formula where Fahrenheit = (kelvin - 273.15) * 9/5 + 32. Next, it adds a new field trans_temp_f at the root level with the Fahrenheit value. Lastly, preserve all existing fields from the parent nexset (each process taken in the canvas). As shown in Figure 5.3.2 is the transformation of python codes.



```
Transform: Code #1 » Python  Transform  Python

1  def transform(input, metadata, args):
2      output = input.copy()
3
4      main = input.get('main') or {}
5      kelvin = main.get('temp') if hasattr(main, 'get') else None
6
7      fahrenheit = None
8      if kelvin is not None:
9          try:
10             kelvin_num = float(kelvin)
11             fahrenheit = round((kelvin_num - 273.15) * 9.0 / 5.0 + 32.0, 2)
12         except:
13             fahrenheit = None
14
15     output['trans_temp_f'] = fahrenheit
16     return output
```

Figure 5.3.2 Transformation python code

The current data flow of weather from the OpenWeather is as shown in Figure 5.3.3. Previously, both of the destination Snowflake Weather Data Sink was active simultaneously. But, Express AI managed to give three options regarding how to update the sink configuration, whether to add a new sink to transformed newset, replace existing newset, or keep both sinks. A good data engineer practice is to not have duplicate data. Therefore, the second option is chosen, where replace existing newset. That is why the destination snowflake weather data sink from newset 1 (OpenWeather Current Weather) is paused, only the snowflake weather data sink from newset 2 (OpenWeather Current Weather-Temp F) is active.

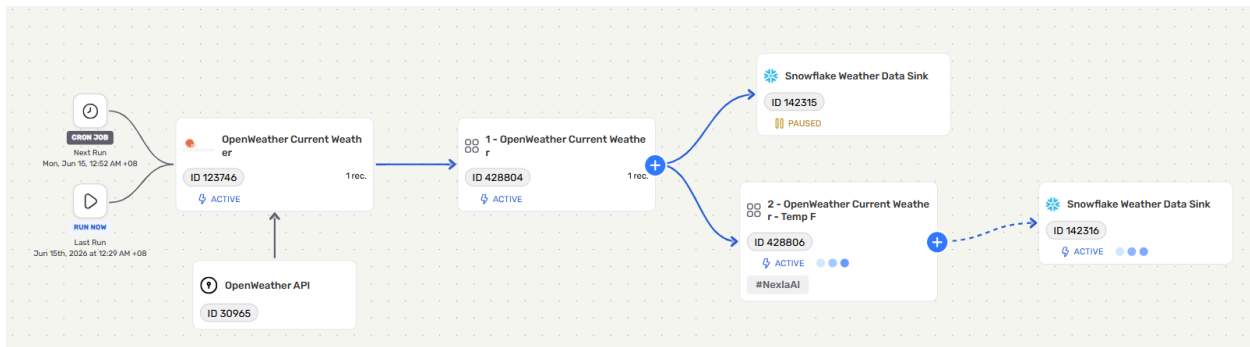


Figure 5.3.3 Current data flow of weather

To make sure the data is transformed and exists, the data is previewed in the Snowflake database as shown in Figure 5.3.4.

	SUNRISE	SUNSET	TRANS_TEMP_F
14	1970-01-21T14:50:08.577Z	1970-01-21T14:51:08.319Z	70.18
15	1970-01-21T14:50:08.577Z	1970-01-21T14:51:08.319Z	70.77
16	1970-01-21T14:50:08.577Z	1970-01-21T14:51:08.319Z	70.43
17	1970-01-21T14:50:08.577Z	1970-01-21T14:51:08.319Z	70.45
18	1970-01-21T14:50:08.577Z	1970-01-21T14:51:08.319Z	70.45
19	1970-01-21T14:50:08.577Z	1970-01-21T14:51:08.319Z	70.25
20	1970-01-21T14:50:08.577Z	1970-01-21T14:51:08.319Z	70.45
21	1970-01-21T14:50:08.577Z	1970-01-21T14:51:08.319Z	70.52
22	1970-01-21T14:50:08.577Z	1970-01-21T14:51:08.319Z	70.54
23	1970-01-21T14:50:08.577Z	1970-01-21T14:51:08.319Z	70
24	1970-01-21T14:50:08.577Z	1970-01-21T14:51:08.319Z	70.23

Figure 5.3.4 Trans_temp_f data existed

6.0 CONCLUSION RESULT

This tutorial successfully demonstrated the construction of a complete AI-assisted ETL data pipeline using Express AI on the Nexla platform. Interacting with the AI agent through natural language prompts enables extracting real-time weather data from the OpenWeatherMap API, applying a data transformation to convert temperature values from Kelvin to Fahrenheit, and loading the resulting dataset into a Snowflake cloud data warehouse, all without writing manually configuring infrastructure.

The pipeline was structured into three stages. In the extraction stage, the AI agent automatically handled API credential validation, schema inference, and source configuration for the OpenWeatherMap Current Weather endpoint. In the transformation stage, Express AI generated a Python function to compute a new derived column, `trans_temp_f`, by applying the standard Kelvin-to-Fahrenheit conversion formula: $\text{Fahrenheit} = (\text{Kelvin} - 273.15) \times 9/5 + 32$. In the loading stage, the transformed Nexset was mapped and written to the `WEATHER_DATA` table in Snowflake under the `nexla_db` database, with the pipeline scheduled to run automatically at regular intervals.

The final output confirmed that the `trans_temp_f` column was successfully populated with Fahrenheit temperature values, as verified through the Snowflake Data Preview interface. The pipeline canvas showed a clear data lineage from the OpenWeatherMap source through the transformation node to the active Snowflake sink.

Overall, the results validate that AI-assisted tools such as Express AI can significantly accelerate pipeline development by abstracting repetitive configuration tasks. However, the exercise also revealed that technical knowledge remains essential, particularly for troubleshooting API key activation delays, provisioning cloud data warehouse resources, and iterating on prompts when the AI agent does not produce the expected output on the first attempt. AI agents serve best as co-pilots, not replacements, for skilled data engineers.

7.0 REFERENCES

Andreas Kretz. (2025, November 5). *I built this complete ETL data pipeline with an AI Agent!* [Video].

YouTube. <https://www.youtube.com/watch?v=qGu7hDaJG1A>

Mattia, D. P., Zhong, M., Sripada, Y., & Jaspars, M. (2025). FlowETL: an autonomous Example-Driven pipeline for data engineering. *ArXiv.org*. <https://doi.org/10.48550/arxiv.2507.23118>

Reis, J., Housley, M., Bill Inmon, Jordan Tigani, Bruno Aziza, Andy Petrella, Sarah Krasnik, Veronika Durgin, Chris Tabb, Jon King, & Kevin Hu. (n.d.). *Fundamentals of Data Engineering*.

Steiner, A., & Bizer, C. (2026). Automatic End-to-End Data Integration using Large Language Models. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2603.10547>

8.0 REFLECTION

YASMIN BATRISYIA BINTI ZAHIRUDDIN (A23CS0201)

After completing the tutorial 5 (ETL data pipeline with an AI Agent), building ETL pipelines with an AI agent looks effortless in a video but takes real understanding to do it on our own. The challenge that encountered was to switch the weather source from WeatherAPI.com to OpenWeather because the agent couldn't validate the API key, then setting up the database and warehouse in Snowflake. Next challenge is where the Express AI agent kept on reporting that it was successfully writing the data, while Snowflake showed the column empty. This means to always verify at the source of truth rather than trusting a tool's own claim of success. Overall, this tutorial emphasizes that low code doesn't mean removing the need for technical knowledge. This is because troubleshooting depends on identifying which layer of the pipeline is failing, and that the diagnostic process is the most valuable learning outcome of this tutorial.

NURUL ADRIANA BINTI KAMAL JEFRI (A23CS0258)

In the tested approach, an AI agent was able to dramatically reduce the time needed to construct a minimum viable pipeline by generating boilerplate connector code and mapping the JSON keys, but was not a replacement for the basic data engineering skills. It continues to be important to have a solid knowledge of the network boundaries, host configuration, and provisioning of warehouses, in order to steer the AI safely and detect execution errors. The point was emphasized during development, as the AI ended up not realizing that it was trying to store a "deprecated" data sink with a new "transformation fork" in the database, incurring duplicate writes. Future use will require optimization of the use of AI - with strict "Idempotency Clauses" to force existing pipelines to be refactored, and automated data observability and schema drift monitoring to maintain data quality over the long-term.

SYARIFAH DANIA BINTI SYED ABU BAKAR (A23CS0183)

Completing this tutorial offered a firsthand look at how AI-assisted orchestration tools like Express AI streamline data pipelines by translating natural language into functional ETL components. While the visual canvas made abstract concepts like data lineage and end-to-end data flows concrete, the execution exposed critical friction points, particularly during the transformation stage. Prompting the agent to convert temperature from Kelvin to Fahrenheit proved to be an iterative, code-like process that demanded precise guidance, such as explicitly specifying the source field (main.temp), target column name, and mathematical formula, to prevent the agent from making incorrect assumptions. Ultimately, this experience underscored that while AI co-pilots significantly compress development time by automating boilerplate tasks, deep domain knowledge remains indispensable for precise prompt iteration, pipeline debugging, and rigorous data validation.